The structure of the primary key for the relationship set depends on the mapping cardinality of the relationship set. As an illustration, consider the entity sets *customer* and *account*, and the relationship set *depositor*, with attribute *access-date*, in Section 2.1.2. Suppose that the relationship set is many to many. Then the primary key of *depositor* consists of the union of the primary keys of *customer* and *account*. However, if a customer can have only one account—that is, if the *depositor* relationship is many to one from *customer* to *account*—then the primary key of *depositor* is simply the primary key of *customer*. Similarly, if the relationship is many to one from *account* to *customer*—that is, each account is owned by at most one customer—then the primary key of *depositor* is simply the primary key of *account*. For one-to-one relationships either primary key can be used.

For nonbinary relationships, if no cardinality constraints are present then the superkey formed as described earlier in this section is the only candidate key, and it is chosen as the primary key. The choice of the primary key is more complicated if cardinality constraints are present. Since we have not discussed how to specify cardinality constraints on nonbinary relations, we do not discuss this issue further in this chapter. We consider the issue in more detail in Section 7.3.

2.4 Design Issues

The notions of an entity set and a relationship set are not precise, and it is possible to define a set of entities and the relationships among them in a number of different ways. In this section, we examine basic issues in the design of an E-R database schema. Section 2.7.4 covers the design process in further detail.

2.4.1 Use of Entity Sets versus Attributes

Consider the entity set *employee* with attributes *employee-name* and *telephone-number*. It can easily be argued that a telephone is an entity in its own right with attributes *telephone-number* and *location* (the office where the telephone is located). If we take this point of view, we must redefine the *employee* entity set as:

- The *employee* entity set with attribute *employee-name*
- The *telephone* entity set with attributes *telephone-number* and *location*
- The relationship set *emp-telephone*, which denotes the association between employees and the telephones that they have

What, then, is the main difference between these two definitions of an employee? Treating a telephone as an attribute *telephone-number* implies that employees have precisely one telephone number each. Treating a telephone as an entity *telephone* permits employees to have several telephone numbers (including zero) associated with them. However, we could instead easily define *telephone-number* as a multivalued attribute to allow multiple telephones per employee.

The main difference then is that treating a telephone as an entity better models a situation where one may want to keep extra information about a telephone, such as

its location, or its type (mobile, video phone, or plain old telephone), or who all share the telephone. Thus, treating telephone as an entity is more general than treating it as an attribute and is appropriate when the generality may be useful.

In contrast, it would not be appropriate to treat the attribute *employee-name* as an entity; it is difficult to argue that *employee-name* is an entity in its own right (in contrast to the telephone). Thus, it is appropriate to have *employee-name* as an attribute of the *employee* entity set.

Two natural questions thus arise: What constitutes an attribute, and what constitutes an entity set? Unfortunately, there are no simple answers. The distinctions mainly depend on the structure of the real-world enterprise being modeled, and on the semantics associated with the attribute in question.

A common mistake is to use the primary key of an entity set as an attribute of another entity set, instead of using a relationship. For example, it is incorrect to model *customer-id* as an attribute of *loan* even if each loan had only one customer. The relationship *borrower* is the correct way to represent the connection between loans and customers, since it makes their connection explicit, rather than implicit via an attribute.

Another related mistake that people sometimes make is to designate the primary key attributes of the related entity sets as attributes of the relationship set. This should not be done, since the primary key attributes are already implicit in the relationship.

2.4.2 Use of Entity Sets versus Relationship Sets

It is not always clear whether an object is best expressed by an entity set or a relationship set. In Section 2.1.1, we assumed that a bank loan is modeled as an entity. An alternative is to model a loan not as an entity, but rather as a relationship between customers and branches, with *loan-number* and *amount* as descriptive attributes. Each loan is represented by a relationship between a customer and a branch.

If every loan is held by exactly one customer and is associated with exactly one branch, we may find satisfactory the design where a loan is represented as a relationship. However, with this design, we cannot represent conveniently a situation in which several customers hold a loan jointly. To handle such a situation, we must define a separate relationship for each holder of the joint loan. Then, we must replicate the values for the descriptive attributes *loan-number* and *amount* in each such relationship. Each such relationship must, of course, have the same value for the descriptive attributes *loan-number* and *amount*.

Two problems arise as a result of the replication: (1) the data are stored multiple times, wasting storage space, and (2) updates potentially leave the data in an inconsistent state, where the values differ in two relationships for attributes that are supposed to have the same value. The issue of how to avoid such replication is treated formally by *normalization theory*, discussed in Chapter 7.

The problem of replication of the attributes *loan-number* and *amount* is absent in the original design of Section 2.1.1, because there *loan* is an entity set.

One possible guideline in determining whether to use an entity set or a relationship set is to designate a relationship set to describe an action that occurs between

entities. This approach can also be useful in deciding whether certain attributes may be more appropriately expressed as relationships.

2.4.3 Binary versus n-ary Relationship Sets

Relationships in databases are often binary. Some relationships that appear to be nonbinary could actually be better represented by several binary relationships. For instance, one could create a ternary relationship *parent*, relating a child to his/her mother and father. However, such a relationship could also be represented by two binary relationships, *mother* and *father*, relating a child to his/her mother and father separately. Using the two relationships *mother* and *father* allows us record a child's mother, even if we are not aware of the father's identity; a null value would be required if the ternary relationship *parent* is used. Using binary relationship sets is preferable in this case.

In fact, it is always possible to replace a nonbinary (n-ary, for $n > 2$) relationship set by a number of distinct binary relationship sets. For simplicity, consider the abstract ternary ($n = 3$) relationship set R, relating entity sets A, B, and C. We replace the relationship set R by an entity set E, and create three relationship sets:

- R_A, relating E and A
- R_B, relating E and B
- R_C, relating E and C

If the relationship set R had any attributes, these are assigned to entity set E; further, a special identifying attribute is created for E (since it must be possible to distinguish different entities in an entity set on the basis of their attribute values). For each relationship (a_i, b_i, c_i) in the relationship set R, we create a new entity e_i in the entity set E. Then, in each of the three new relationship sets, we insert a relationship as follows:

- (e_i, a_i) in R_A
- (e_i, b_i) in R_B
- (e_i, c_i) in R_C

We can generalize this process in a straightforward manner to n-ary relationship sets. Thus, conceptually, we can restrict the E-R model to include only binary relationship sets. However, this restriction is not always desirable.

- An identifying attribute may have to be created for the entity set created to represent the relationship set. This attribute, along with the extra relationship sets required, increases the complexity of the design and (as we shall see in Section 2.9) overall storage requirements.
- A n-ary relationship set shows more clearly that several entities participate in a single relationship.

- There may not be a way to translate constraints on the ternary relationship into constraints on the binary relationships. For example, consider a constraint that says that R is many-to-one from A, B to C; that is, each pair of entities from A and B is associated with at most one C entity. This constraint cannot be expressed by using cardinality constraints on the relationship sets R_A, R_B, and R_C.

Consider the relationship set *works-on* in Section 2.1.2, relating *employee*, *branch*, and *job*. We cannot directly split *works-on* into binary relationships between *employee* and *branch* and between *employee* and *job*. If we did so, we would be able to record that Jones is a manager and an auditor and that Jones works at Perryridge and Downtown; however, we would not be able to record that Jones is a manager at Perryridge and an auditor at Downtown, but is not an auditor at Perryridge or a manager at Downtown.

The relationship set *works-on* can be split into binary relationships by creating a new entity set as described above. However, doing so would not be very natural.

2.4.4 Placement of Relationship Attributes

The cardinality ratio of a relationship can affect the placement of relationship attributes. Thus, attributes of one-to-one or one-to-many relationship sets can be associated with one of the participating entity sets, rather than with the relationship set. For instance, let us specify that *depositor* is a one-to-many relationship set such that one customer may have several accounts, but each account is held by only one customer. In this case, the attribute *access-date*, which specifies when the customer last accessed that account, could be associated with the *account* entity set, as Figure 2.6 depicts; to keep the figure simple, only some of the attributes of the two entity sets are shown. Since each *account* entity participates in a relationship with at most one instance of *customer*, making this attribute designation would have the same meaning

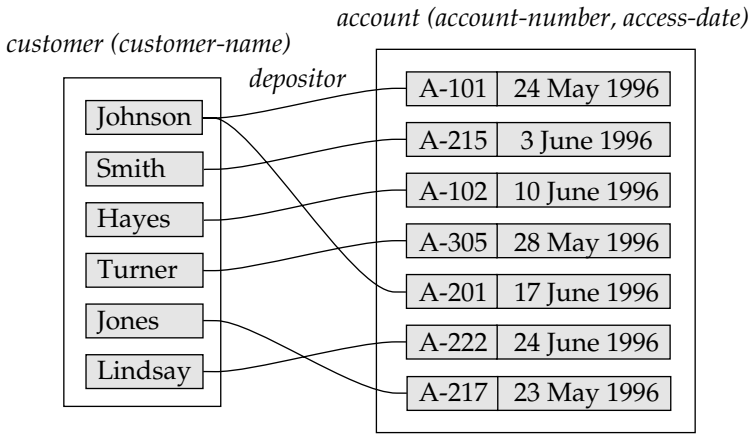


Figure 2.6 *Access-date* as attribute of the *account* entity set.

as would placing *access-date* with the *depositor* relationship set. Attributes of a one-to-many relationship set can be repositioned to only the entity set on the "many" side of the relationship. For one-to-one relationship sets, on the other hand, the relationship attribute can be associated with either one of the participating entities.

The design decision of where to place descriptive attributes in such cases—as a relationship or entity attribute—should reflect the characteristics of the enterprise being modeled. The designer may choose to retain *access-date* as an attribute of *depositor* to express explicitly that an access occurs at the point of interaction between the *customer* and *account* entity sets.

The choice of attribute placement is more clear-cut for many-to-many relationship sets. Returning to our example, let us specify the perhaps more realistic case that *depositor* is a many-to-many relationship set expressing that a customer may have one or more accounts, and that an account can be held by one or more customers. If we are to express the date on which a specific customer last accessed a specific account, *access-date* must be an attribute of the *depositor* relationship set, rather than either one of the participating entities. If *access-date* were an attribute of *account*, for instance, we could not determine which customer made the most recent access to a joint account. When an attribute is determined by the combination of participating entity sets, rather than by either entity separately, that attribute must be associated with the many-to-many relationship set. Figure 2.7 depicts the placement of *access-date* as a relationship attribute; again, to keep the figure simple, only some of the attributes of the two entity sets are shown.

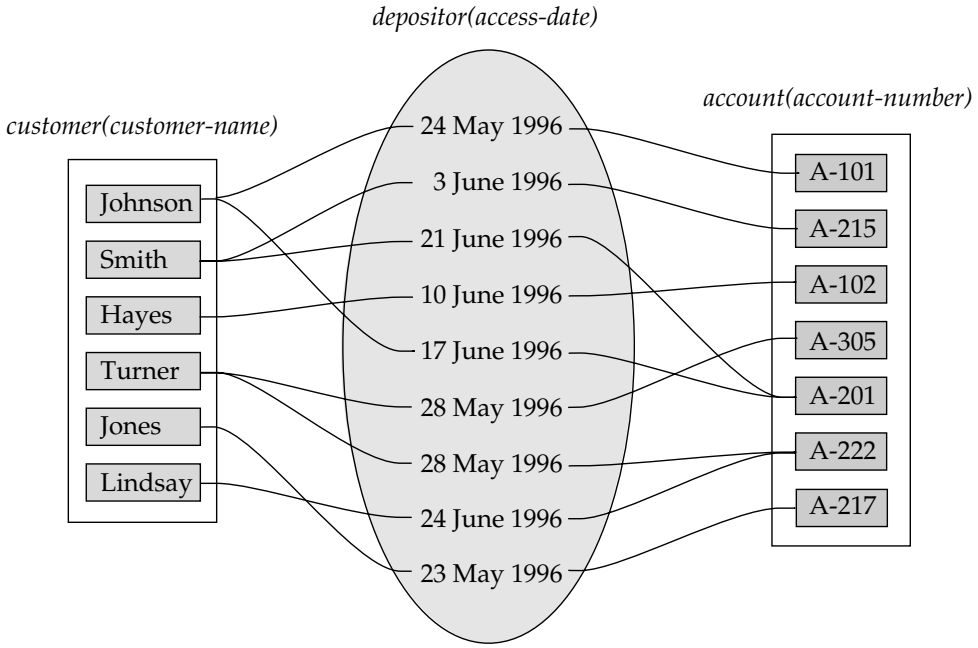


Figure 2.7 *Access-date* as attribute of the *depositor* relationship set.